

Manual de Construção — Radar de Pregões 24/7

Engenharia de software, integração com APIs governamentais, motor de polling assíncrono e guia de reconstrução passo a passo

o. Visão Geral da Arquitetura

O **Radar de Pregões 24/7** foi projetado para operar como um sentinela ininterrupto. Ele resolve um problema crítico da equipe de licitações: prazos relâmpago de envio de anexos, convocações em sala de disputa e publicações de recursos que exigem manifestação técnica em poucas horas.

O sistema foi implementado em **Node.js puro** (sem dependências externas de pacotes npm) e **PowerShell COM** para integração com o Outlook desktop. A ausência de bibliotecas externas pesadas garante portabilidade instantânea em qualquer ambiente Windows corporativo.

MÓDULO / ARQUIVO	TECNOLOGIA	PAPEL NO SISTEMA
app/compras-api.js	Node.js (https nativo)	Conector de APIs REST públicas para Compras.gov.br (PNCP) e Portal de Compras Públicas. Normaliza UASGs, busca certames e extrai frases do chat do pregoeiro.
app/server.js (loop 24/7)	Node.js assíncrono	Executa varreduras periódicas a cada 1 minuto (INTERVALO_MIN), compara a assinatura de estado de cada certame e orquestra notificações.
data/acompanhados.json	JSON em disco	Banco de dados transacional leve com a lista de pregões cadastrados, histórico de eventos de cada um e lista de responsáveis por e-mail.
scripts/enviar-alerta-auto.ps1	PowerShell + Outlook COM	Instancia o processo do Outlook clássico de forma oculta e despacha e-mails com cabeçalho corporativo e tag [B2G-RADAR-NTSEC] .
app/ui.html (Aba Vigilância)	HTML5 / CSS3 / Vanilla JS	Painel responsivo com KPIs, feed em tempo real de mensagens de chat, modal de responsáveis e formulário multiportal.

ROTEIRO DAS FASES DE RECONSTRUÇÃO

1. O conector de APIs públicas (compras-api.js)
2. O modelo de dados e assinatura de integridade
3. O motor de polling contínuo sem bloqueio de I/O
4. O mecanismo de despacho de e-mails via Outlook COM
5. As rotas de controle no servidor Node.js
6. A interface visual, KPIs e feed de atos oficiais

Fase 1 — O Conector de APIs Públicas

Objetivo: criar o módulo que consome dados das APIs do governo federal e municipais sem autenticação e sem bibliotecas externas.

PROMPT PARA COLAR NA IA

Crie o modulo app/compras-api.js em Node.js puro (usando apenas o modulo nativo "https").
O arquivo deve conter funcoes para:

1. normalizaUasg(uasg): limpa espacos e pontuacao, garantindo que seja string numerica de ate 6 digitos.
2. normalizaNumero(numero): extrai numeros e trata formatos de pregao como "PE 12/2026" ou "12".
3. consultarPregao({uasg, numero, ano}): faz GET na API do Compras.gov/PNCP:
https://dadosabertos.compras.gov.br/modulo-pesquisa-pregao/1_consultarPregao_PNCP_14133?pagina=1&tamanhoPagina=10&numeroUasg={uasg}&anoCompra={ano}&numeroCompra={numero}
Caso nao encontre na Lei 14.133, tenta fallback na API legado da Lei 8.666.
4. consultarChatPregao(codigoLicitacao): extrai as mensagens e avisos do pregoeiro.
5. consultarPCP(item) e consultarChatPCP(codigo): consulta certames no Portal de Compras Publicas (api.portaldecompraspublicas.com.br).

Trate erros com timeout de 8 segundos para evitar travamento. Exporte todas as funcoes.

VERIFICAÇÃO

Execute uma consulta de teste via linha de comando para uma UASG conhecida (ex.: 70001). O retorno deve trazer um objeto JSON estruturado contendo órgão, objeto, valor estimado e situação cadastral.

Fase 2 — Modelo de Dados e Assinatura de Estado

Objetivo: persistir o estado dos pregões e calcular um hash de assinatura para saber exatamente quando houve mudança real.

PROMPT PARA COLAR NA IA

Adicione em `compras-api.js` a funcao `assinatura(registro)`:

Gere uma string hash compacta combinando:

- situacao cadastral (ex.: Publicada, Em Disputa, Suspensa)
- data e hora de abertura ou encerramento
- quantidade de itens homologados
- quantidade e texto da ultima frase do chat

Isso serve para detectar mudancas reais no pregão sem depender de comparacoes campo a campo demoradas.

Defina a estrutura de `data/acompanhados.json`:

Array de objetos contendo:

```
{
  "id": "70001-12-2026",
  "portal": "comprasgov",
  "uasg": "70001", "numero": "12", "ano": "2026",
  "apelido": "Nome do Órgão",
  "responsaveis": ["email1@grupontsec.com"],
  "dataCadastro": "ISO...", "ultimaChecagem": "ISO...",
  "assinatura": "hash...",
  "mudouAgora": false,
  "registro": { ...dados brutos },
  "eventos": [ { "quando": "ISO...", "texto": "...", "novo": true, "alertaEnviado": false } ]
}
```

VERIFICAÇÃO

Ao consultar o mesmo pregão duas vezes seguidas sem que haja novos atos oficiais, o hash da assinatura deve ser rigorosamente idêntico.

Fase 3 — O Motor de Polling Contínuo

Objetivo: implementar a rotina de varredura periódica em segundo plano no servidor web sem bloquear o atendimento de requisições HTTP.

PROMPT PARA COLAR NA IA

No `app/server.js`, implemente o loop assíncrono de vigilancia:

1. Uma variavel global `VARREDURA` controlando status (rodando, ultima, comMudanca, verificados, ultimoErro).

2. Funcao assincrona `varrerVigilancia(motivo)`:

- Le `data/acompanhados.json`. Se estiver vazio, retorna imediatamente.
- Itera pelos itens com pausa de 400ms entre requisicoes para nao sobrecarregar as APIs governamentais.
- Chama `comprasApi` para atualizar cada item.
- Se a nova assinatura for diferente da anterior:
 - * Identifica quais eventos sao novos (mudanca de status ou mensagens de chat inéditas).
 - * Marca `mudouAgora = true` no item e `novo = true` no evento.
 - * Chama `dispararAlertaEmail(item, mudancas)`.
- Salva `data/acompanhados.json` e `data/historico-eventos.json`.

3. No arranque do servidor, execute uma varredura inicial apos 3 segundos, e programe um `setInterval` a cada 1 minuto (`INTERVALO_MIN * 60 * 1000`) com `t.unref()` para nao segurar o processo.

ATENÇÃO

O uso de `t.unref()` no timer e pausas com `setTimeout` entre requisições é obrigatório. Sem isso, se a rede oscilar ou a API do governo demorar para responder, o Node.js pode travar todas as outras abas do aplicativo.

Fase 4 — Despacho de E-mails via Outlook COM

Objetivo: enviar mensagens de notificação corporativa formatadas em HTML através do Outlook do usuário.

PROMPT PARA COLAR NA IA

```
Crie o script scripts/enviar-alerta-auto.ps1:
- Parametro de entrada: -JsonPath com o caminho de um arquivo temporario contendo { to, subject, html }.
- Instancia o Outlook COM: New-Object -ComObject Outlook.Application.
- Cria mensagem $mail = $outlook.CreateItem(0).
- Preenche $mail.To, $mail.Subject, $mail.HTMLBody.
- Executa $mail.Send().
- No server.js, crie dispararAlertaEmail(item, mudancas):
  * Monta template HTML com destaque visual vermelho/ambar, lista de mudancas, tabela de dados da licitacao e link direto.
  * Grava JSON temporario e invoca enviar-alerta-auto.ps1 via spawn em background.
  * Marca alertaEnviado = true nos eventos despachados para nunca duplicar envio.
```

VERIFICAÇÃO

Cadastre seu próprio e-mail num pregão de teste e clique no botão de disparo de alerta manual. Uma notificação estruturada deve chegar na sua caixa de entrada em segundos com a tag `[B2G-RADAR-NTSEC]`.

Fase 5 e 6 — Rotas da API e Interface de Usuário

Objetivo: expor as operações via HTTP REST e montar o painel visual com KPI cards e feed de chat.

PROMPT PARA COLAR NA IA

```
1. Adicione ao server.js as rotas:
- GET /api/acompanhados: retorna lista e status da varredura.
- POST /api/acompanhar: valida e cadastra novo pregão.
- POST /api/desacompanhar: remove do monitoramento.
- POST /api/chechar-status: força varredura manual imediata.
- POST /api/vigilancia-marcas-lido: remove os alertas pendentes.
- POST /api/vigilancia-atualizar-responsaveis: altera e-mails vinculados.
2. No app/ui.html, crie o container #modVigilancia:
- Grade superior de 4 KPIs (Pregões, Movimentação Não Lida, Valor Total, Status 24/7).
- Painel mestre-detelhe: coluna esquerda com a lista de cards de certames monitorados; coluna direita com cabeçalho completo, botões de ação e linha do tempo de mensagens (estilo chat de mensageria).
- Formulario expansivel com abas para Compras.gov, Licitações-e e PCP.
```

Gotchas Técnicas e Soluções Adotadas

DESAFIO TÉCNICO	CAUSA REAL	SOLUÇÃO DEFINITIVA
Licitações-e (Banco do Brasil) sem dados automáticos	O portal é construído em JSF antigo com estado de sessão no servidor e protegido por Cloudflare invisível.	O sistema cadastra o pregão com link direto para consulta rápida em um clique e permite registro manual de eventos.
Frases do pregoeiro repetidas a cada ciclo	O chat do Compras.gov devolve o histórico completo a cada chamada, sem paginação de novas mensagens.	O sistema armazena a coleção de hashes de texto das frases já processadas em <code>historico-eventos.json</code> , descartando repetições.
Corrupção do arquivo JSON em desligamento	Quedas de energia ou encerramento forçado do Node durante o salvamento de	A gravação é feita via <code>writeFileSync</code> atômico após o ciclo completo de varredura.

